

EDUNEXUS

A Multidimensional Educational Data Science Framework

FINAL PROJECT REPORT

B.Tech | Artificial Intelligence and Data Science

ANALYTICAL FOUNDATION	VERIFIED PROJECT SCOPE
UCI Dataset 697	4,424 student records; 37 original columns
Educational data science	Profiling, quality, statistics and explainable analysis
M1-M14	Core analytical modules, digital twin and simulation

Prepared from the supplied project documentation
and source-code snapshot

22 September 2026

ABSTRACT

Educational institutions hold heterogeneous records describing entry qualifications, academic performance, financial circumstances and progression. These records become useful for educational decisions only when their meanings, quality and analytical limitations are understood. EduNexus implements a multidimensional educational data science workflow that connects these activities through a shared analytical dataset and modular Python services.

The current project uses the UCI Machine Learning Repository dataset **Predict Students' Dropout and Academic Success**, Dataset ID 697. The supplied raw file contains 4,424 student records and 37 columns: 36 explanatory attributes and one academic outcome. The outcomes are Graduate, Enrolled and Dropout. A standardized analytical master adds an anonymous record identifier, five readable category labels and eight engineered features, producing 51 columns. The workflow covers dataset profiling, semantic integration, quality assessment, cleaning audits, descriptive statistics, exploratory analysis, hypothesis testing, feature engineering, semester progression, cohort discovery, anomaly analysis and educational interpretation. K-Means supports exploratory segmentation; PCA supports two-dimensional visualization; Isolation Forest identifies unusual multivariate records. These algorithms support the data science narrative rather than define a supervised prediction project.

Fourteen modules expose this workflow. The Student Digital Twin provides a deterministic, population-relative analytical profile. The Intervention Simulator recalculates selected academic indicators under hypothetical changes. Python, Pandas, NumPy, SciPy and scikit-learn implement the analytical work; FastAPI exposes JSON endpoints; React, TypeScript, TanStack React Query and Recharts present the outputs.

This report distinguishes design intent from observed implementation. The supplied code contains limitations in cleaning execution, quality scoring, insight generation, identity handling and simulation validation. These are disclosed instead of being converted into unsupported claims. Reported dataset counts and stored audit results are traceable to the supplied files. Unreported statistical coefficients, clustering scores and predictive accuracy are not invented. EduNexus is therefore presented as an implemented analytical research framework whose outputs require contextual interpretation and further validation before institutional use.

Keywords: educational data science; learning analytics; data quality; exploratory data analysis; statistical inference; feature engineering; student cohorts; anomaly detection; analytical digital twin; what-if analysis.

Reading and evidence conventions

References [1]-[3] identify the supplied instructions, foundation document and project archive. References [4]-[12] provide primary methodological documentation. A source-code implementation establishes what a function does; it does not establish that its output is educationally valid. File-derived counts, source-defined parameters, stored audit outputs, targeted verification outcomes and unreported analytical results are identified separately throughout the report. No author names, registration numbers, supervisor endorsements or approval certificates have been fabricated.

CONTENTS

ABSTRACT	2
CHAPTER 1 - INTRODUCTION	4
CHAPTER 2 - DATASET AND DATA UNDERSTANDING	6
CHAPTER 3 - SYSTEM ARCHITECTURE	9
CHAPTER 4 - DATA PREPROCESSING	12
CHAPTER 5 - DATA SCIENCE METHODOLOGY	14
CHAPTER 6 - MODULE IMPLEMENTATION	19
CHAPTER 7 - TECHNOLOGY STACK	27
CHAPTER 8 - RESULTS AND DISCUSSION	29
CHAPTER 9 - TESTING AND VALIDATION	32
CHAPTER 10 - LIMITATIONS AND FUTURE SCOPE	35
CHAPTER 11 - CONCLUSION	38
REFERENCES	39
APPENDIX A - VERIFIED API INVENTORY	40
APPENDIX B - SOURCE TRACEABILITY	42

CHAPTER 1 - INTRODUCTION

1.1 Background

Educational data science combines data management, statistical reasoning, visualization and domain interpretation to study educational processes. A grade, an admission score or a scholarship flag is an observation. It becomes evidence only after the analyst establishes what it measures, checks how it was collected and compares it with a suitable reference. A student's current grade, for example, does not independently describe progression, completion efficiency or similarity to other students.

EduNexus addresses this problem by bringing several analytical perspectives into one reproducible software structure. The project does not treat a chart as an explanation by itself. Its intended chain begins with a known research dataset, proceeds through understanding and preparation, and ends with interpretable educational observations. The dashboard is the presentation layer through which a user inspects that chain. Core statistics, transformations and algorithms reside in Python rather than in React components. [2], [3]

The framework is multidimensional because it considers academic, demographic, entry-related, socioeconomic and macroeconomic attributes together. These dimensions permit richer descriptive questions than marks alone, while also requiring care: a financial association is not a causal finding, a cluster is not a diagnosis, and an unusual record is not automatically a student at risk.

1.2 Problem statement

Raw educational tables do not provide an integrated way to assess quality, describe distributions, test associations, compare progression and inspect individual records in context. An isolated average can hide variation, a visual relationship can be misleading, and an engineered indicator can be difficult to defend unless its formula is explicit. Disconnected tools further obscure which source and transformation produced a displayed value.

The project problem is to create a modular analytical framework that transforms student records into understandable, traceable educational evidence. The framework must preserve variable meaning, expose the assumptions of its methods and separate observed outcomes from exploratory indicators. It must also make its outputs accessible through an interactive interface without moving statistical responsibility into the interface.

1.3 Motivation

The first motivation is methodological completeness. Profiling, quality checks and descriptive analysis are necessary before more complex methods can be interpreted. The second is explainability: an analyst should be able to trace a grade-change value to two semester measurements or explain why standardization precedes distance-based clustering. The third is practical accessibility. Faculty or student-support users may benefit from a clear chart and evidence summary even when they do not directly operate a Python notebook.

A final motivation is responsible interpretation. Educational records concern people whose circumstances cannot be reduced to a score. The project supports investigation and discussion; it does not establish automated disciplinary decisions, causal intervention effects or a validated dropout probability.

1.4 Objectives

Objective	Implementation focus
Establish a shared data foundation	Load the UCI source, standardize headers, decode selected categories and expose one analytical master.
Make data reliability visible	Profile missingness, duplicates and selected domain violations; display cleaning audit information.
Explain population characteristics	Compute descriptive statistics, distributions, group summaries and visual comparisons.
Investigate associations	Provide Pearson, Spearman, Welch and Chi-square procedures with explicit hypotheses.
Derive interpretable indicators	Implement eight documented features with transparent formulas and rounding conventions.
Compare progression and cohorts	Analyse two observed semesters and discover standardized K-Means groupings.
Inspect unusual records	Use Isolation Forest to prioritize multivariate review without equating rarity with failure.
Support individual exploration	Compare population percentiles and recalculate hypothetical academic scenarios.

Table 1.1. Project objectives and their concrete analytical responsibilities. Some objectives are partially implemented; the relevant limitations are documented in Chapters 4, 6, 8 and 9.

1.5 Scope

The report covers the UCI-based M1-M14 analytical implementation in the outer project directory of the supplied archive. It includes its data schema, Python engines, FastAPI routes, React views and confirmed configuration files. The analytical population is the supplied historical research dataset, not a live feed from Chennai Institute of Technology or another local institution. Temporal analysis means comparison between Semester 1 and Semester 2, not continuous attendance monitoring or a daily learning-management-system event stream.

The archive also contains older OULAD assets, an embedded project copy, CIT-Sentinel materials, a Node-based application service and an ML Hub extension. Their existence is acknowledged, but their results and claims are not substituted for UCI-based EduNexus evidence. Supervised prediction is outside the principal report scope. The future SENTINAL D / EduPredict AI layer remains conceptually separate from the data science foundation requested here. [1]-[3]

1.6 Contributions

The implemented contribution is an integrated analytical workflow with a shared dataset and clearly separated computational and presentation layers. Eight explicit transformations make progression, completion and financial status available for downstream inspection. Complementary statistical and unsupervised methods allow population, group and individual views of the same records. The digital twin and simulator extend inspection to relative individual profiles and deterministic scenarios.

These are engineering and analytical contributions within this project, not claims that K-Means, Isolation Forest, percentile profiles or dashboards are new algorithms. A further contribution of the report is its implementation traceability: formulas, route names and limitations are reconciled against source code so that the academic explanation remains defensible.

CHAPTER 2 - DATASET AND DATA UNDERSTANDING

2.1 Dataset source

The primary dataset is **Predict Students' Dropout and Academic Success**, published through the UCI Machine Learning Repository as Dataset ID 697. The official repository describes student-level data assembled from a higher education institution, including entry information and academic performance after the first and second semesters. The repository lists 4,424 instances and 36 features; adding the outcome produces the 37 original columns present in the supplied file. [4]

The current loader reads `data/raw/uci_predict_students_dropout_and_academic_success.csv`. It does not require the older OULAD tables found elsewhere in the archive. The report's dataset identity follows the current loader and the explicit user instructions. [1], [3]

2.2 Dataset description

Property	Verified interpretation
Unit of observation	One student record.
Raw records	4,424.
Original columns	37, comprising 36 explanatory variables and the Target column.
Outcome values	Graduate, Enrolled and Dropout.
Prepared representation	51 columns: 37 originals, one generated ID, five readable labels and eight engineered features.
Semester coverage	Academic summaries for two semesters.
Primary storage	Raw CSV and processed master CSV; Parquet writing is attempted by the builder.
Auxiliary extended CSV	Retained separately because no shared UCI entity key is established.

Table 2.1. Raw and analytical dataset structures, verified against the uploaded CSV files and `uci_loader.py`. The 51-column master is not a dataset with 51 independent original measurements.

The generated identifier has the form `EDU-000001`. It is assigned from row position. It is stable while source order remains unchanged, but it is not a persistent institution-issued identity and should not be used to join unrelated files. Reordering the source changes the correspondence between identifier and record.

2.3 Feature categories

Category	Included variables	Analytical role
Demographic	Marital status, nationality, gender, age at enrollment, international flag	Describe composition and examine subgroup patterns with appropriate privacy and fairness safeguards.
Application and entry	Application mode/order, course, attendance schedule, previous qualification and grade, admission grade	Describe entry pathways and preparation.
Socioeconomic	Parents' qualifications and occupations, displaced status, special needs, debtor, tuition status, scholarship	Inspect support-related associations without assigning causal explanations.
Semester 1	Credited, enrolled, evaluated, approved and unevaluated units; semester grade	Establish the first observed academic position.
Semester 2	Corresponding second-semester measures	Compare completion and performance over two observations.
Macroeconomic	Unemployment, inflation and GDP-related values	Provide contextual variables; repeated contextual values are not independent student-level exposures.
Outcome	Target	Describe historical outcome composition and compare groups.

Table 2.2. Semantic domains defined in backend/data/schema.py. Numeric storage does not make all codes quantitative measurements.

A course code is a category identifier, so averaging course codes has no educational interpretation. Likewise, the daytime/evening field describes a schedule category, not a student's attendance percentage. This distinction prevents the UCI attributes from being incorrectly described as live classroom attendance or digital engagement data.

2.4 Target variable

The target records the observed academic outcome as Graduate, Enrolled or Dropout. The Enrolled group must remain distinct: it is not equivalent to either successful completion or failure. The target can support outcome-conditioned summaries and retrospective comparisons. It is excluded from the numerical input lists used for current K-Means clustering and Isolation Forest.

However, the cohort-labeling rules use cluster-level outcome proportions after clustering. The resulting names therefore contain retrospective outcome information. They should not be described as independently discovered prospective risk labels or interpreted as validated predictions for newly enrolled students.

2.5 Class distribution

Outcome	Count in supplied raw CSV	Share of all records
Graduate	2,209	49.93%
Dropout	1,421	32.12%
Enrolled	794	17.95%
Total	4,424	100.00%

Table 2.3. Counts obtained directly from the supplied Target column; shares are count divided by 4,424, rounded to two decimals. These are file-derived descriptive values, not classifier performance.

The classes are unequal in size. This matters when comparing group counts: a larger group can contribute more records without having a larger within-group outcome rate. The foundation document's approximate percentages are replaced by these reproducible counts and a single consistent rounding convention. The proportions describe this dataset and must not be reported as current rates at the student's college.

2.6 Dataset selection rationale

The dataset provides heterogeneous dimensions within a manageable student-level table. It supports meaningful profiling, numerical distributions, categorical cross-tabulation, statistical comparisons and interpretable feature derivation. Paired semester summaries make progression analysis possible, while variation across academic and entry attributes supports exploratory clustering and multivariate anomaly detection.

The selection is therefore justified by analytical coverage and interpretability, not simply by record count. Publicly documented variables also make the workflow easier to inspect and reproduce. The repository's original classification purpose does not require EduNexus to become a prediction-centered project: the same observations can support descriptive and exploratory questions. [2]-[4]

2.7 Data characteristics and constraints

The supplied raw CSV contains zero Pandas-detected missing cells and zero exact duplicate rows. These checks were repeated directly on the raw file, before a synthetic ID could conceal duplicates. The stored cleaning audit also reports zero missing values handled and zero enrollment-limit violations. This supports a clean structural starting point; it does not establish absence of measurement error, selection bias or semantic ambiguity.

The data combines continuous measurements, counts and coded categories. There are only two semester summaries per student and no current institutional event feed in the main UCI workflow. Grades and completed-unit counts may reflect both academic performance and participation; a zero should not be interpreted as missing without a source-supported rule. Historical outcomes also limit prospective claims. Semester 2 measurements would be unavailable to a system intended to act before Semester 2, creating a timing problem for any future early-warning predictor.

CHAPTER 3 - SYSTEM ARCHITECTURE

3.1 Overall architecture

EduNexus separates data access, analytical computation, HTTP delivery and interactive presentation. The Python layer loads the shared master, and module-specific functions calculate outputs when requested. FastAPI routers expose these functions through JSON responses. React views retrieve the responses and display charts, tables and explanations. [3]

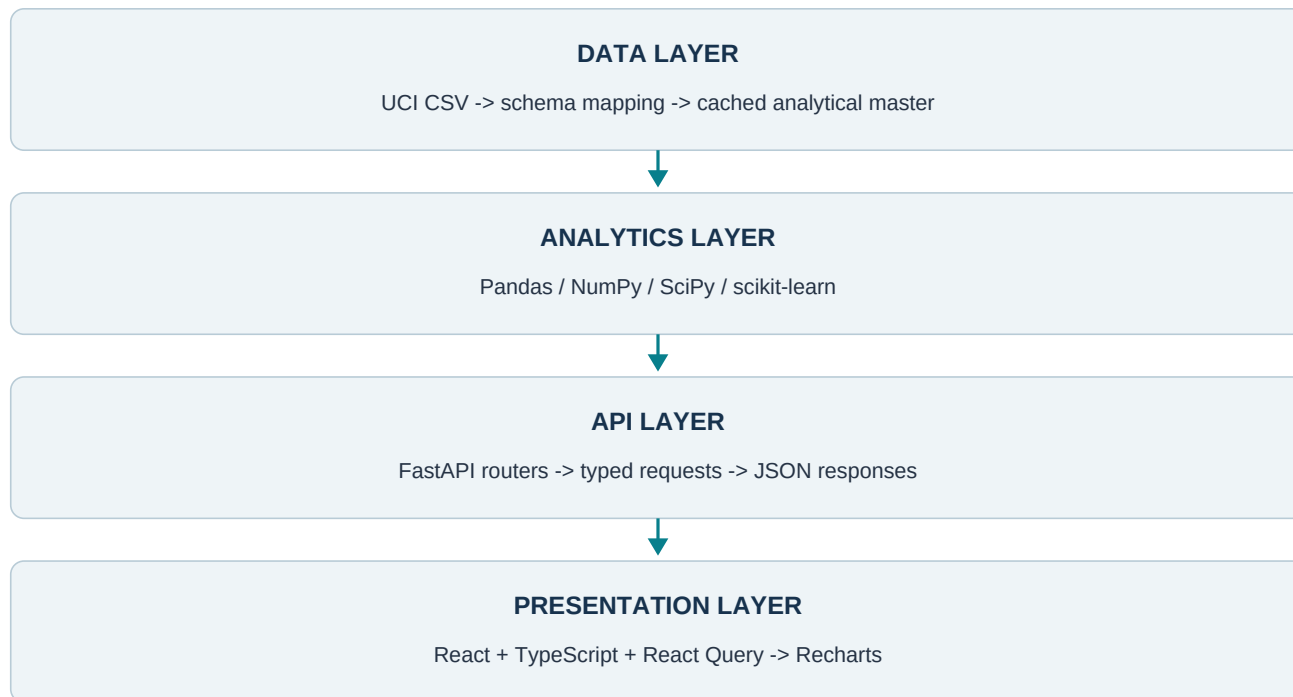


Figure 3.1. Implemented layer separation. The dataset feeds the Python analytical services; FastAPI transports their outputs; the frontend presents them. Arrows indicate the principal data dependency, not a deployed streaming system.

3.2 Data layer

`backend/data/schema.py` defines raw-to-canonical header mappings and semantic domains. `uci_loader.py` reads the raw CSV, adds row-based anonymous identifiers, decodes selected categories and computes the eight features. It attempts a Parquet export and writes a CSV master. `get_master_dataset()` first reuses an in-process DataFrame cache, then an existing processed CSV, and otherwise builds the master.

This reuse reduces repeated loading, but it does not include source-hash or schema-version validation. A processed file can therefore become stale after source changes. Also, the accessor's existing-file path reads the CSV rather than the Parquet file. Parquet presence alone is not proof that a request used Parquet-backed loading.

3.3 Analytics layer

The `backend/analytics/` package contains distinct functions for quality, cleaning, descriptive analysis, EDA, statistical tests, feature summaries, progression, cohorts, anomalies, insights, digital twins and simulations. Pandas handles tabular operations, NumPy supports numerical arrays and histogram bins, SciPy supplies statistical procedures, and scikit-learn supplies clustering, scaling, projection and isolation-based detection.

Most engines independently call the shared dataset accessor. They do not pass a succession of newly cleaned DataFrames through all fourteen modules. The pipeline is therefore an analytical dependency structure and user workflow; it is not an enforced execution scheduler. This distinction is necessary because, for example, the master builder computes features before the user opens the quality or cleaning screens.

3.4 API layer

backend/main.py creates the FastAPI application with version 3.0.0 and registers module routers. A route accepts path, query or body parameters, invokes the corresponding Python function and returns a JSON-compatible dictionary. For example, GET /api/cohorts/summary?k=4 executes the cohort engine. POST /api/statistical_analysis/run accepts the test type and selected variables.

Query validation constrains cluster count to 2-8 and contamination to 0.01-0.30. Pydantic models declare statistical and simulation request fields. Error handling is route-specific rather than uniform. Some functions return HTTP 400 for invalid selections, while several wrappers convert general exceptions into HTTP 500. The health and pipeline-status routes provide metadata; the latter uses fixed completion values rather than executing checks.

3.5 Presentation layer

React views correspond to the analytical modules. TanStack React Query maintains asynchronous query state and cached responses, while Fetch performs HTTP requests. Recharts renders analytical outputs such as bars, scatter plots, lines and radar profiles. Tailwind CSS and shared UI components provide layout and styling. TypeScript describes selected response shapes and component contracts, although some parts of the code remain loosely typed.

The Vite development server runs on port 8080 and proxies /api to http://127.0.0.1:8000. This allows module views to use relative API URLs. A production deployment needs equivalent routing; a development proxy does not automatically accompany a static build. The older Node service used by other application areas is separate from this analytics API.

3.6 End-to-end data flow

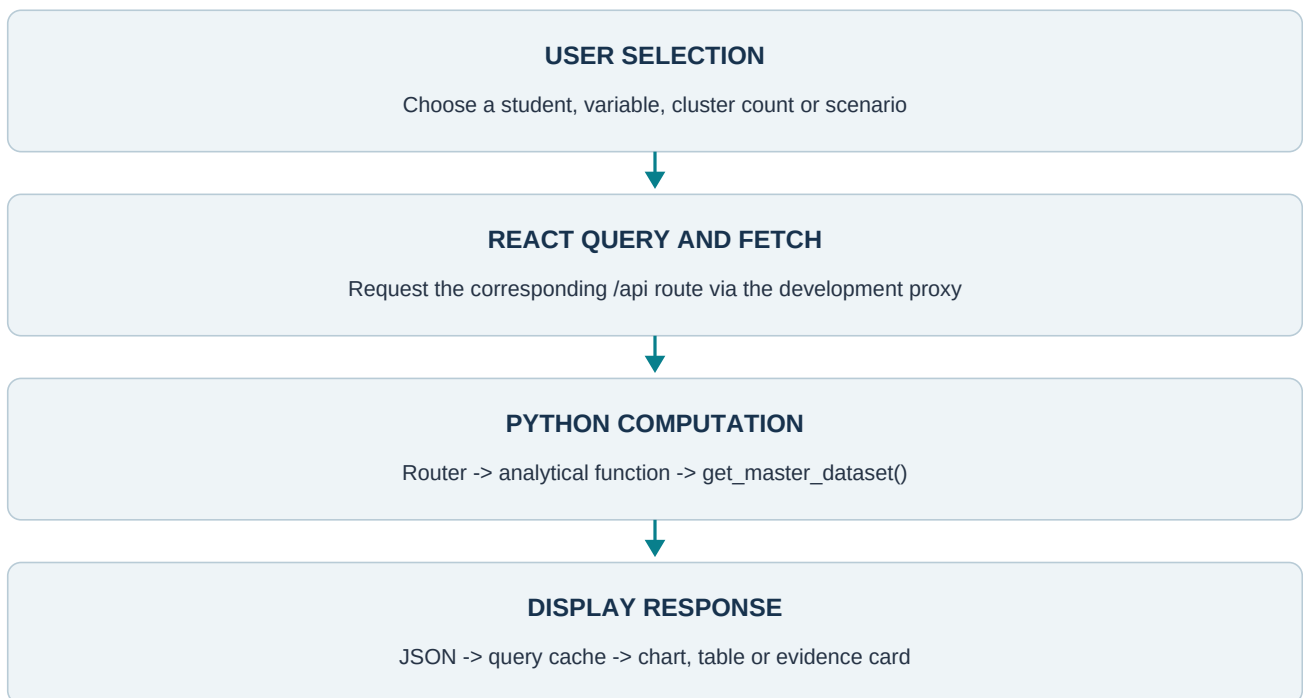


Figure 3.2. Request-response path for the analytical interface. The query cache manages server data; it does not replace Python computation.

A concrete flow starts when the analyst opens the feature engineering view. The view requests GET /api/features/summary. FastAPI calls run_feature_engineering_summary(), which retrieves the master DataFrame. The engine selects the eight catalogued features, computes their descriptive summaries and returns the

formula catalog and distributions. React receives the JSON and renders the supplied values. These results can then guide the analyst's choices in progression, cohort or individual-profile views.

For a student scenario, the frontend selects a generated identifier and posts proposed second-semester values to `/api/intervention-simulator/simulate`. The simulator obtains the baseline twin, recalculates approval and progression indicators, compares them with the unchanged population distribution and returns before-versus-scenario radar values. No change is committed to the underlying student record.

3.7 Software architecture and pipeline interpretation



Figure 3.3. Conceptual data science workflow. Each stage has a technical contract described in Chapter 6. In the current code, many stages read the same prepared master independently; the arrows express analytical order rather than mandatory execution gates.

The architecture is modular because a statistical function can be inspected separately from its router and chart. A visualization can change without redefining the statistical formula. A new data adapter can be introduced without requiring every React component to parse raw files. Nevertheless, central imports create coupling: `main.py` also imports the ML Hub, whose trainer requires XGBoost and SHAP even though these are not listed in the supplied Python requirements. That extension can affect startup of the core application and should be isolated or declared explicitly.

CHAPTER 4 - DATA PREPROCESSING

4.1 Dataset profiling

Profiling establishes the input's structure before interpretation. M1 reports row and column counts, Pandas dtypes, missing percentages, cardinality, numerical minimum/maximum/mean, target frequencies and semantic domains. This makes it possible to distinguish a numerical measurement from an encoded category and to identify unsuitable variables before selecting an analysis.

The current explorer profiles the prepared master rather than an untouched raw table. Its reported 51 variables include generated labels, an identifier and engineered features. For that reason, Chapter 2 separately records the raw 37-column structure. Profiling results are inputs to method selection: a numerical grade can enter a correlation, while a category label can define a contingency table.

4.2 Data quality assessment

M3 evaluates completeness, uniqueness and selected domain rules on the master. Completeness is computed from column missing percentages. Uniqueness is based on exact-row duplication. Validity checks age in [15,90], admission grade in [0,200], approved units not exceeding enrolled units in either semester, and membership of the target in its permitted set.

$$Q = 0.30 C + 0.30 U + 0.30 V + 0.10 K$$

Here C, U and V are the implemented completeness, uniqueness and validity components. **K is assigned the constant 98.5 in the current code.** The composite is therefore partly configured and must not be described as an independently measured overall quality result. Validity subtracts summed violation counts, so a record failing multiple rules can be counted more than once. The procedure is a useful diagnostic summary but not a comprehensive data-quality standard.

4.3 Missing values

Pandas `isnull()` provides the missing-cell audit, and numerical analyses commonly use `dropna()` or fill numerical inputs with zero. The raw and stored master files have no Pandas-detected missing cells. This does not prove that all values are substantively known: placeholder codes, empty semantic categories or invalid numerical values require separate domain checks.

A distinct issue occurs when enrolled units are zero. The master builder replaces a zero denominator with NaN, computes approved/enrolled and fills the result with zero. This prevents division errors but maps an undefined rate to the same numerical value as a genuine zero-completion rate. A future schema should preserve an explicit denominator-validity flag or an unknown rate when the educational interpretation requires it.

4.4 Duplicate handling

The current audit calls `df.duplicated()` after the synthetic `student_id` has been inserted. Unique IDs can make otherwise identical original records appear distinct. The supplied raw CSV was therefore independently checked without generated IDs, confirming zero exact duplicates in this snapshot.

The M4 response uses field names such as `duplicates_removed` and `rows_after`, but its function does not call `drop_duplicates()` or persist a revised master. In this implementation those fields describe audit-derived values, not demonstrated deletion operations. A production cleaner should compare original business attributes, define retention rules, log row identities and rebuild dependent features after any removal.

4.5 Data type validation

Header normalization strips surrounding whitespace and maps known names to canonical `snake_case`. The mapping handles the spelling `Nacionality` and the trailing tab in the `daytime/evening` header. Selected binary fields are mapped to readable labels such as `Yes/No` and `Up to Date/In Arrears`.

The schema file documents semantics but does not impose a complete executable dtype contract at ingestion. Pandas inference remains responsible for parsing. Several unknown category mappings fall back to `No` or `In Arrears`, which can collapse invalid inputs into substantive categories. Type enforcement, permitted-code validation and explicit unknown labels are therefore required extensions rather than completed protections.

4.6 Cleaning implementation

`run_data_cleaning()` reads the master, counts duplicates, missing cells and approval-limit violations, constructs a list of operations and writes `cleaning_audit_log.json`. Header and label standardization occur earlier in the master builder. The cleaner itself does not implement a configurable imputation strategy, removal step, range cap or cleaned-master rewrite despite its broader descriptive comments.

This distinction matters educationally: a report should say what was performed, not infer operations from a module title. For the supplied already-complete file, the saved audit reports 4,424 rows before and after, zero duplicates removed, zero missing values handled and zero invalid records flagged. It does not demonstrate how the system behaves on an intentionally dirty dataset.

4.7 Data preparation for analysis

Preparation adds the identifier and readable labels, derives eight indicators, and exposes the resulting `DataFrame` to the analytical engines. K-Means standardizes its selected feature matrix; Isolation Forest uses its selected numerical columns without an explicit scaler. Selected statistical analyses drop rows missing either requested variable. These are method-specific preparations, not a universal normalization of every column.

The intended handoff is a consistent, interpretable table whose raw fields and derived fields can be distinguished. Current preparation achieves repeatable formulas for the supplied snapshot, while stronger schema enforcement, cache invalidation and execution-aware lineage remain necessary. The report preserves these boundaries throughout the module descriptions.

CHAPTER 5 - DATA SCIENCE METHODOLOGY

5.1 Descriptive statistics

Descriptive statistics summarize what is present in a dataset without claiming a causal mechanism or predicting an unseen outcome. M5 uses Pandas and SciPy to calculate measures of center, spread and shape for a selected numerical variable. For categorical variables it produces frequencies, percentages and the mode. [3]

Measure	Definition or formula	When and why it is used
Mean	$\bar{x} = \sum(x_i) / n$	Summarizes the arithmetic center; inspect with spread because extreme observations affect it.
Median	Middle ordered value or mean of the two middle values	Gives a robust center when a distribution is asymmetric.
Sample variance	$s^2 = \sum((x_i - \bar{x})^2) / (n - 1)$	Quantifies dispersion in squared units; Pandas defaults to sample variance.
Standard deviation	$s = \sqrt{s^2}$	Expresses dispersion in the original unit, such as grade points.
Quartiles	Q1 = 25th percentile; Q3 = 75th percentile	Describe the central half of the observations.
Interquartile range	IQR = Q3 - Q1	Provides a spread measure less driven by extreme tails.
Skewness	Standardized third central moment, as calculated by SciPy	Describes distribution asymmetry; interpret alongside the histogram.
Excess kurtosis	Standardized fourth central moment minus 3, as calculated by SciPy	Describes tail-related shape relative to a normal reference; not a direct failure-risk measure.

Table 5.1. Principal descriptive methods. The engine also returns mode, minimum, maximum and coefficient of variation. Default SciPy skewness and kurtosis use their standard bias settings unless explicitly changed.

The implemented univariate outlier rule flags values below $Q1 - 1.5 \text{ IQR}$ or above $Q3 + 1.5 \text{ IQR}$. A flag identifies a value outside a statistical fence; it does not prove a data-entry error. Valid unusual records should not be removed automatically. The engine computes ten histogram bins and rounds displayed statistics, so verification should compare underlying calculations before display rounding where possible.

Coefficient of variation is calculated as $100s/\bar{x}$, with zero returned when the mean equals zero. A zero-mean safeguard avoids division failure but does not make the coefficient meaningful for signed change variables or values whose mean is close to zero. Descriptive measures must be selected according to semantic measurement type, not merely Pandas dtype.

5.2 Exploratory data analysis

M6 investigates distributions and relationships before formal testing. Histograms show numerical concentration and tails. Boxplot summaries compare quartiles and spread across outcome groups. Scatter plots inspect the relationship between the two semester grades. Grouped counts compare scholarship or tuition categories with outcomes. A correlation matrix summarizes pairwise linear relationships among nine selected numerical fields.

The implementation returns admission-grade summaries by target, a seeded sample of up to 300 scatter records, scholarship/outcome counts and tuition/outcome counts. Custom bivariate analysis samples up to 400 records.

Sampling improves display responsiveness, but a scatter plot of the sample is not the entire population. Aggregated counts and the correlation matrix are calculated from their full selected inputs.

Plots generate questions rather than establish explanations. A visible association between payment status and outcome, for example, motivates careful cross-tabulation and testing. It does not identify whether finance, program characteristics or another unmeasured factor caused the outcome. The feature set also contains algebraically related quantities; correlations involving grade change and its component grades partly reflect mathematical construction.

5.3 Correlation analysis

Pearson correlation measures standardized linear covariation. It ranges from -1 to +1, with sign indicating direction and magnitude describing linear association. M7 calls `scipy.stats.pearsonr` and returns its coefficient and p-value. It is suitable for meaningful numerical measurements, while severe outliers, constant inputs and nonlinear structures require attention. [5]

$$r = \text{sum}((x_i - \bar{x})(y_i - \bar{y})) / \sqrt{\text{sum}((x_i - \bar{x})^2) \text{sum}((y_i - \bar{y})^2)}$$

Spearman correlation compares ranks and measures monotonic association. It can capture a consistent increasing or decreasing relationship without requiring a straight line. The implementation calls `scipy.stats.spearmanr`, which handles tied ranks. It is still sensitive to study design and is not a causal estimator. For tied or small samples, inferential interpretation needs additional care. [6]

The current code labels coefficient magnitudes as negligible below 0.1, small below 0.3, medium below 0.5 and large otherwise. These are software conventions, not universal educational thresholds. Report the coefficient and context alongside any category. The EDA correlation matrix is descriptive; it does not itself supply hypothesis tests for every cell.

5.4 Hypothesis testing

A null hypothesis H_0 specifies the reference claim being examined. The alternative H_1 describes the departure of interest. The p-value measures how incompatible the observed statistic is with H_0 under the test's assumptions; it is not the probability that H_0 is true. The engine compares p with $\alpha = 0.05$ by default. The current request model does not expose α even though the underlying function accepts it.

Welch's independent-samples t-test. To compare the mean of a numerical measure across two independent groups, the engine calls `ttest_ind(..., equal_var=False)`. Unlike the equal-variance form, Welch's method accommodates unequal group variances. For example, Semester 1 grades may be compared across the two scholarship labels. This is an association between groups; it does not estimate the causal effect of receiving a scholarship. [7]

$$t = (\text{mean}_1 - \text{mean}_2) / \sqrt{s_1^2/n_1 + s_2^2/n_2}$$

The engine also calculates a pooled-standard-deviation Cohen's d, which expresses mean separation in standardized units. Its sign depends on group order. A small p-value and a small effect can coexist when sample size is large. The implementation's magnitude labels for d are below 0.2: Small; below 0.8: Medium; otherwise: Large. These particular bins should be reported as code-defined labels, not attributed to a universal convention.

$$d = (\text{mean}_1 - \text{mean}_2) / s_{\text{pooled}}$$

$$s_{\text{pooled}} = \sqrt{((n_1-1)s_1^2 + (n_2-1)s_2^2) / (n_1+n_2-2)}$$

Chi-square independence test. For two categorical variables, Pandas creates a contingency table and SciPy calculates the Chi-square statistic, p-value and degrees of freedom. H_0 states that the variables are independent. Expected counts arise from the row and column margins. The test requires adequate expected counts and independent observations; the engine does not return expected counts for review. [8]

```
chi_square = sum_over_cells((Observed - Expected)^2 / Expected)
```

A critical implementation constraint is that the t-test selects the first two observed groups if more than two exist. Using the three-class target as its grouping variable silently excludes one class while the reported sample size counts all pairwise-complete records. A defensible analysis must deliberately choose a binary grouping or implement explicit group selection. Welch's independent test is also not the correct method for comparing two semester measurements from the same students; a paired procedure would be a separate extension. Multiple-testing adjustment and confidence intervals are not provided by the current route.

5.5 Feature engineering

Eight engineered features are defined in both the loader and the feature catalog. They convert raw measurements into directly interpretable analytical quantities. Let G1 and G2 denote Semester 1 and Semester 2 grades; A1 and A2 approved units; E1 and E2 enrolled units; D the debtor flag; and T the tuition-up-to-date flag. The formulas below follow the implemented code, including its denominator and rounding choices. [3]

5.5.1 semester_grade_change

```
semester_grade_change = round(G2 - G1, 2)
```

This signed difference measures the direction and size of change between the two observed grades. It improves analysis by distinguishing students with equal Semester 2 grades but different prior performance. It feeds progression thresholds, cohort inputs and anomaly inputs. A negative value describes decline, not its cause.

5.5.2 semester_approval_change

```
semester_approval_change = A2 - A1
```

This count difference compares completed units across semesters. It adds a completion perspective that grades alone may miss. Interpretation must consider changing enrollment loads: a lower approved count is not necessarily worse efficiency if fewer units were enrolled. Unlike the approval rates, this indicator is not divided by a denominator.

5.5.3 approval_rate_sem1

```
approval_rate_sem1 = round(A1 / E1, 4), with zero returned when E1 = 0
```

This ratio describes the proportion of Semester 1 enrolled units approved. It supports comparison of students with different loads and contributes a population percentile to the digital twin's returned data. The zero-denominator convention is computational, not a claim that undefined completion equals failed completion.

5.5.4 approval_rate_sem2

```
approval_rate_sem2 = round(A2 / E2, 4), with zero returned when E2 = 0
```

This ratio provides the second-semester completion measure and scales the progression index. It is also a radar dimension through its population percentile. The master formula does not clip rates to [0,1]; quality validation should detect approvals above enrollments. The simulator uses a different clipping convention, which is documented under M14.

5.5.5 academic_progression_index

```
academic_progression_index = round((0.4 G1 + 0.6 G2) approval_rate_sem2, 2)
```

The indicator weights the later semester more heavily and then adjusts the weighted grade by Semester 2 completion. It combines performance and completion in an explainable composite. The weights 0.4 and 0.6 are design choices fixed in the code, not estimated optimal weights or validated predictors. It supports cohort analysis and relative student comparison while retaining the need to inspect its separate components.

5.5.6 financial_risk_indicator

`financial_risk_indicator = 1 if D = 1 and T = 0; otherwise 0`

The feature marks the conjunction of debtor status and tuition arrears. Its operational meaning is a compound financial-status flag. The label 'risk' does not establish dropout likelihood, creditworthiness or causality. It can guide a financial-support enquiry, provided a human reviews the underlying records and avoids punitive conclusions.

5.5.7 academic_load_ratio

`academic_load_ratio = round(E1 / 6.0, 2)`

This feature expresses Semester 1 load relative to a six-unit reference. It makes load easier to interpret as a ratio, but six is an application-defined baseline rather than a verified universal full-time requirement across all programs. Program-specific interpretation is needed before calling a student overloaded or underloaded.

5.5.8 performance_stability

`performance_stability = round(abs(G2 - G1), 2)`

This is an absolute two-semester change. Lower values indicate more similar grades across the two observations. It is not statistical variance or a longitudinal stability estimate over many semesters, despite broader wording in a catalog comment. The absolute value removes direction, so it should be inspected alongside signed grade change; a large improvement and large decline can have the same value.

All eight stored feature columns were recomputed from the supplied master inputs and matched the formulas within numerical tolerance during report verification. This confirms formula consistency for this snapshot. It does not validate the indicators as measures of educational success or intervention effectiveness.

5.6 Temporal and progression analytics

M9 compares the two observed semester positions. It classifies a rounded grade change above +0.5 as Improving, below -0.5 as Declining, and within the inclusive interval [-0.5,+0.5] as Stable. The threshold is a transparent operational rule rather than a learned boundary or statistical significance test.

The engine returns category counts and percentages, semester averages, average approvals and evaluations, outcome-group trajectories and an eight-bin grade-change histogram. These outputs show whether the selected population exhibits different directions of change. They do not establish the timing of change within a semester, its cause or a future trajectory. The current progression endpoint does not return a complete per-student approval-rate trajectory even though the master contains both rates.

5.7 Cohort analytics

K-Means groups records by minimizing within-cluster squared Euclidean distances. M10 uses eight selected numerical fields: admission grade, age at enrollment, both semesters' approved units and grades, signed grade change, and the progression index. StandardScaler transforms each dimension so that different units do not dominate distance simply because their numerical scales differ. [3], [9]

`z_ij = (x_ij - mean_j) / scale_j`

`K-Means objective = sum_over_clusters sum_over_members ||z_i - center_c||^2`

The default is four clusters, with `random_state=42` and `n_init=10`. The API permits two through eight clusters. Four is a configured starting value, not a demonstrated optimal cohort count. Cluster sizes, means and outcome proportions support interpretation. Silhouette and Davies-Bouldin scores are calculated to summarize separation and compactness, but no score is presented as a confirmed report result unless it appears in the supplied evidence.

Clustering is appropriate here because the purpose is exploratory grouping rather than learning labels from a target. Nevertheless, overlapping derived features can weight similar academic information more than once. Sensitivity analysis across feature choices and cluster counts is needed before using the groupings for support allocation.

5.8 Principal component analysis

PCA produces orthogonal directions that capture variance in the standardized matrix. The implementation retains two components to display cohort points in a plane and reports their combined explained variance. It applies PCA after forming the standardized input matrix; K-Means uses that full matrix rather than the two displayed coordinates. [10]

A two-dimensional projection cannot preserve every distance or distinction in the original space. Apparent overlap on the chart need not imply identical full-dimensional profiles. The chart samples up to 400 records using an unseeded NumPy selection, so displayed points can change between calls even though the K-Means seed is fixed. The report does not equate visual separation with proven educational usefulness.

5.9 Anomaly detection

Isolation Forest repeatedly partitions a feature space with random splits. Records that can be isolated quickly tend to receive more unusual scores. M11 uses nine numerical inputs covering entry grade, age, both semesters' enrolled and approved units and grades, and signed grade change. It fills missing numerical inputs with zero and sets `random_state=42`. [3], [11]

The default contamination is 0.05. This parameter influences the decision threshold; it is not an observed prevalence of educational problems and not a measured error rate. The API allows values from 0.01 to 0.30. Output includes flagged-record counts, a score histogram and at most 50 highest-ranked anomalous records.

Univariate IQR detection asks whether one value is extreme relative to its marginal distribution. Multivariate detection asks whether a combination is unusual in the selected feature space. A record may be individually ordinary on several columns yet unusual in combination. Conversely, a valid high achiever can be rare. The correct next action is contextual review, not an automatic failure label.

The engine negates the decision function for record ranking, but its histogram uses the original decision-function sign. This sign distinction must be retained in chart labels. Textual reasons are deterministic heuristics, not a feature-attribution explanation of the Isolation Forest itself.

5.10 Educational insight generation

M12 constructs six deterministic evidence cards covering outcome composition, scholarship association, a first-semester comparison, progression, the largest cohort and multivariate anomalies. It combines computed outputs with text templates. The intended benefit is to connect a numerical result with a readable educational question while retaining the source module and supporting metrics.

The implementation requires correction before every card can be described as evidence-backed. It selects scholarship groups using labels that do not match the master's Yes/No values, producing undefined percentages. It hardcodes a p-value statement and interprets a scholarship-based grade comparison as evidence about subsequent academic difficulty, which the selected test does not establish. These are specific source-verification findings, not a reason to invent substitute results.

A defensible insight states the population, variables, method, sample size, result and limitations. A progression card can describe observed decline and suggest review; it cannot attribute decline to fatigue without evidence. The twin and simulator likewise summarize or recalculate indicators without claiming causal effects.

CHAPTER 6 - MODULE IMPLEMENTATION

Each module below is documented through its input, processing method, implementation, output and downstream use. This establishes the technical contract for every stage in the conceptual workflow. The contracts also identify where the supplied source implements a narrower behavior than the foundation document describes. Source filenames are relative to the outer EDUNEXUS/ project. [2], [3]

6.1 M1 - Dataset Explorer

Purpose and why it matters. M1 establishes what the analytical table contains before the user interprets any chart. It prevents confusion between original attributes, generated fields and numerical category codes. An analyst can use it to decide whether a field is suitable for a numerical or categorical procedure.

Input. The prepared master returned by `get_master_dataset()`, the domain and description dictionaries, and the compatibility metadata for the extended CSV. Although this is the first interface module, its input already contains the generated identifier, labels and engineered features.

Processing and method. The loader adapter selects columns by Pandas dtype, counts non-null and unique values, calculates missing percentages and derives numerical minimum, maximum and mean. It assigns semantic domains by checking the schema lists and obtains outcome frequencies with `value_counts()`. This is data profiling and descriptive metadata extraction rather than predictive modeling.

Libraries and implementation. Pandas supplies the tabular operations. `backend/data/loaders.py` implements `get_dataset_summary()`. The route `GET /api/datasets/summary` exposes the result, while `GET /api/datasets/status` supplies availability metadata. `DatasetExplorerView.tsx` uses React Query, Fetch and Recharts to display it.

Output and handoff. The response includes record and variable totals, numerical and categorical counts, missing and duplicate counts, domain distribution, target distribution and a column explorer. The analyst uses this metadata to choose quality rules, visualizations and tests. It is not a persisted transformation of the data.

Educational use case and boundary. Before comparing admission grades across outcomes, a faculty analyst checks the field description and available outcome categories. The explorer should not be cited as a raw-file schema report without explaining why it shows 51 rather than 37 columns. Likewise, its master-level duplicate count should be supplemented with a raw-attribute check because generated IDs can mask duplication.

6.2 M2 - Data Integration & Lineage

Purpose and why it matters. M2 describes the provenance and preparation of a common analytical representation. Integration in the current UCI implementation means semantic standardization and enrichment of one primary student table. It does not mean that unrelated files have been successfully joined.

Input. The shared master and the secondary extended-CSV compatibility status. The supporting builder uses the raw UCI CSV, canonical-name mapping and selected category mappings.

Processing and method. The builder renames headers, assigns row-based IDs, decodes five binary categories and derives the eight feature columns. The semantic mapper constructs a six-step lineage description: source ingestion, schema standardization, identifier assignment, category decoding, feature synthesis and secondary-source compatibility assessment. It explicitly retains the extended dataset independently because a shared UCI entity key is unavailable.

Libraries and implementation. Pandas and NumPy implement preparation in `backend/data/uci_loader.py`; `semantic_mapper.py` returns lineage metadata; `integrator.py` is a compatibility adapter. The interfaces are `POST /api/integration/run` and `GET /api/integration/summary`, used by `DataIntegrationView.tsx`.

Output and handoff. M2 returns the primary dataset name, record and column counts, secondary-source status and a lineage list. The CSV master is reused by later modules. Parquet is an attempted additional export, with failures silently ignored by the builder.

Educational use case and boundary. An examiner can trace an academic progression index back to its grade and approval fields rather than treat it as an unexplained score. However, lineage row counts are fixed at 4,424 in the source, and the integration route can reuse an existing cached master. The output is a descriptive lineage record, not a transaction-level execution history with hashes, timestamps and validated row-count transitions. Persistent identity across reordered source versions remains a future improvement.

6.3 M3 - Data Quality Observatory

Purpose and why it matters. M3 makes selected reliability problems visible before statistical interpretation. Its outputs support a decision to inspect or correct records, rather than silently treating every loaded value as valid.

Input. The full analytical master. The procedure assumes expected columns such as age, admission grade, target and both semesters' enrollment and approvals are available.

Processing and method. Pandas counts nulls by column and exact duplicated rows. Boolean filters apply the five validity conditions described in Section 4.2. Component percentages are combined using weights 0.30, 0.30, 0.30 and 0.10. The consistency component is the fixed source constant 98.5, and validity uses summed rule violations rather than unique affected students.

Libraries and implementation. `backend/analytics/quality.py` implements `run_quality_audit()` using Pandas and ordinary numerical operations. `GET /api/quality/summary` returns the audit. `DataQualityView.tsx` renders component summaries and issue information using React Query.

Output and handoff. The JSON includes total records and variables, component scores, a composite, missing values by column, duplicate count and validity issues with rule descriptions and severity. These outputs should guide M4 inspection and qualify later analytical claims. The current architecture does not enforce a quality threshold that blocks downstream routes.

Educational use case and boundary. If approved units exceed enrolled units, the analyst can investigate the source instead of accepting an approval ratio above one. A high displayed composite does not establish correctness of every record. The fixed consistency value, limited rule coverage and identifier-sensitive duplicate test must be disclosed when the score is used in an academic presentation. Testing the audit on deliberately invalid records remains necessary for a stronger validation claim.

6.4 M4 - Data Cleaning Studio

Purpose and why it matters. M4 provides visibility into preparation checks and records an audit artifact. Its current behavior is best described as a cleaning audit interface; its name should not imply that it performs all possible repair operations.

Input. The current master and, for a summary request, an existing `data/processed/cleaning_audit_log.json`. The saved file can be returned without recalculating its checks.

Processing and method. `run_data_cleaning()` counts duplicated rows, missing cells and enrollment-limit violations. It creates records describing schema normalization, duplicate detection, range checking and categorical-label standardization. The function writes this audit dictionary to JSON. The schema and label transformations referenced in the log are performed in the master builder, not newly applied by the cleaning function.

Libraries and implementation. Pandas supplies checks; Python `json` and `pathlib` support the audit file. `backend/analytics/cleaning.py` implements the functions. `GET /api/cleaning/summary` reads or generates an

audit, and `POST /api/cleaning/run` generates it explicitly. `DataCleaningView.tsx` calls the run route with a React Query mutation and refreshes the summary afterward.

Output and handoff. The result reports rows before and after, nominal duplicate-removal and missing-handling counts, flagged records and operation descriptions. It does not return or save a newly repaired master. Downstream modules continue using the existing accessor and cache.

Educational use case and boundary. The stored audit documents that the supplied snapshot has no counted nulls or approval-limit violations. It does not demonstrate successful imputation, deduplication or capping on dirty inputs. A future implementation should separate detected problems, approved actions and actual applied changes, write a new version of the master and invalidate dependent analytical caches. File-write exceptions are currently suppressed, so a success-shaped response does not itself guarantee durable audit persistence.

6.5 M5 - Descriptive Statistics Engine

Purpose and why it matters. M5 characterizes the center, variability, shape and frequency structure of a selected variable. It provides the numerical foundation for EDA and helps the user avoid interpreting an average without its distribution.

Input. A selected canonical column name and the master DataFrame. Missing values in that column are removed for the calculation. The reported sample size therefore refers to non-missing observations for that variable.

Processing and method. For numerical data, Pandas calculates mean, median, mode, sample variance, sample standard deviation, extrema and quartiles. SciPy computes skewness and excess kurtosis. NumPy produces ten histogram bins. The engine calculates IQR fences and counts values beyond them. For categorical data it calculates category counts, shares and the most frequent category.

Libraries and implementation. `backend/analytics/descriptive.py` contains `get_columns_list()` and `compute_variable_statistics()`. Routes are `GET /api/statistics/columns`, `GET /api/statistics/variable/{col_name}` and the aggregate `GET /api/statistics/summary`. `StatisticsView.tsx` requests variable-specific statistics.

Output and handoff. A numerical response contains 13 named statistics, sample size, histogram, outlier count and fences. A categorical response contains cardinality, mode and category frequency records. EDA and hypothesis selection use these distributions to determine which comparisons are meaningful; this is an analyst handoff, not a compulsory call from one module to the next.

Educational use case and boundary. An analyst can compare mean and median semester grades and inspect spread before describing typical performance. The engine will also accept numeric category codes, so semantic judgment remains essential. Constant, empty or very small inputs can produce undefined statistics. A comprehensive version should validate these cases and serialize unavailable statistics explicitly rather than relying on generic JSON behavior.

6.6 M6 - Exploratory Data Analysis

Purpose and why it matters. M6 exposes population structure and candidate relationships visually. It connects descriptive summaries to questions suitable for later testing, without treating visual patterns as formal evidence by themselves.

Input. The master, a fixed list of nine correlation fields and optionally user-selected x, y and grouping columns for bivariate inspection.

Processing and method. Pandas calculates a correlation matrix, admission-grade summaries by outcome and scholarship/tuition outcome counts. A reproducible sample of at most 300 records supplies the standard semester scatter display. The custom bivariate function samples at most 400 records with a fixed seed and converts selected x and y values to floats. This is exploratory multivariate and grouped analysis.

Libraries and implementation. backend/analytics/eda.py provides run_eda_analysis() and run_custom_bivariate(). The routes are GET /api/eda/summary and GET /api/eda/bivariate?x=...&y=...&group=... EDAView.tsx uses React Query and Recharts to display the returned values.

Output and handoff. Outputs include correlation values, grouped five-number-style admission summaries with means and sample sizes, sampled point records, and categorical outcome counts. The analyst can select candidate variable pairs for M7 or inspect which dimensions should be included in cohort analysis.

Educational use case and boundary. A semester scatter plot can show whether similar first-semester grades lead to a wide range of second-semester grades. That variability can motivate further subgroup analysis. It does not identify a cause of decline. The custom route checks column presence but does not fully validate numeric suitability before float conversion. Returned group minima and maxima should also not be confused automatically with Tukey boxplot whiskers. Chart definitions must match the statistics supplied.

6.7 M7 - Statistical Analysis

Purpose and why it matters. M7 turns selected analytical questions into explicit statistical tests. It returns the null hypothesis, alternative, test statistic and decision instead of relying solely on visual impressions.

Input. A request containing test_type, var1 and var2. Supported identifiers are pearson, spearman, t-test and chi-square. The underlying function uses alpha 0.05 unless called directly with another value.

Processing and method. The engine validates that both columns exist, removes pairwise-missing rows and dispatches to SciPy. Pearson and Spearman produce correlation coefficients and p-values. Welch compares the first two encountered groups of a numerical variable and computes Cohen's d. Chi-square uses a Pandas contingency table to examine categorical independence.

Libraries and implementation. backend/analytics/statistical_tests.py uses SciPy, Pandas and NumPy. POST /api/statistical_analysis/run invokes it; GET /api/statistical_analysis/columns lists selectable fields. StatisticalAnalysisView.tsx sends the selected test request using a mutation.

Output and handoff. The response includes the procedure name, variables, statistic, p-value, alpha, sample size, hypotheses, decision and interpretation. Correlation responses add direction and magnitude labels; Welch adds d; Chi-square adds degrees of freedom. The insight layer consumes selected results, but their narrative interpretation must match the actual comparison performed.

Educational use case and boundary. A scholarship-label versus target Chi-square test examines whether observed distributions are associated. It does not show that scholarships caused graduation. The API currently lacks explicit test-type enums, binary-group enforcement, assumption diagnostics and multiple-comparison control. If the target is used for a Welch test, only two of its three classes are selected implicitly. The report therefore explains how to choose a valid question rather than treating every selectable combination as statistically legitimate.

6.8 M8 - Feature Engineering

Purpose and why it matters. M8 makes derived educational indicators transparent. It explains the relationship between a feature's formula and its intended meaning so that downstream users can assess the indicator rather than accept an unexplained score.

Input. The master containing the eight already-computed features and FEATURE_CATALOG, which documents names, formulas, source columns, expected ranges and interpretations.

Processing and method. The underlying construction occurs in uci_loader.py: signed differences, approved/enrolled ratios, a weighted grade-completion index, a logical conjunction flag, a six-unit load ratio and

absolute grade change. The M8 engine then computes each feature's mean, median, standard deviation, minimum and maximum. It does not train a representation or fit formula weights.

Libraries and implementation. Pandas and NumPy implement the transformations and summaries. `backend/analytics/feature_engineering.py` contains the catalog and `run_feature_engineering_summary()`. GET `/api/features/summary` returns them to `FeatureEngineeringView.tsx`.

Output and handoff. The response reports a feature count of eight, the catalog and descriptive distributions. Grade change feeds M9, selected features feed M10/M11, and approval/progression measures support M13/M14. This provides an explicit bridge between original observations and higher-level comparisons.

Educational use case and boundary. Two students with the same final grade may have different signed grade changes and completion ratios, supporting a more informative discussion than a grade-only ranking. Catalog ranges are descriptive expectations, not enforced constraints. The progression weights and load denominator are design choices; performance stability is absolute change rather than variance. No extra 'implemented features' have been invented for this report. Verification found the stored eight-feature values consistent with the source formulas for the uploaded snapshot.

6.9 M9 - Temporal & Progression Analytics

Purpose and why it matters. M9 adds a direction-of-change perspective to static performance. It distinguishes an observed improvement, decline or approximately stable grade profile across the two available semesters.

Input. The master, particularly semester grades, approved units, evaluation counts, outcome and the rounded `semester_grade_change` feature.

Processing and method. Boolean thresholds split students into Improving ($>+0.5$), Declining (<-0.5) and Stable (inclusive -0.5 to $+0.5$). Pandas calculates category counts, percentages, semester-wide averages and outcome-conditioned averages. NumPy divides grade change into eight histogram bins. These are descriptive repeated-observation comparisons, not a forecasting model or a causal trend analysis.

Libraries and implementation. `backend/analytics/progression.py` implements `run_progression_analytics()`. `temporal.py` is a proxy adapter. GET `/api/temporal/summary` exposes the results to `TemporalAnalyticsView.tsx` and also supports the dashboard overview.

Output and handoff. The response contains semester comparison values, trend distribution, progression by outcome and a change histogram. The insight engine uses category percentages to construct a progression card. Individual twin evidence also uses signed grade change, although its wording uses a zero boundary rather than M9's half-point stability band.

Educational use case and boundary. A user can identify records worth closer review after a decline and compare those patterns with completion counts. The available data does not support weekly trajectories, attendance velocity or real-time intervention windows. Exact values at ± 0.5 are classified Stable. Sensitivity to that threshold should be studied before it becomes an operational support rule. Causes such as fatigue or workload cannot be inferred solely from the two grade summaries.

6.10 M10 - Cohort Analytics

Purpose and why it matters. M10 reduces a heterogeneous population into exploratory groups of similar numerical profiles. It helps an analyst inspect patterns shared by groups while retaining the option to examine individual records.

Input. Eight selected academic and entry variables from the master and a cluster count k , default four. Missing numerical values are filled with zero before scaling.

Processing and method. StandardScaler standardizes the selected matrix. K-Means fits k clusters with a fixed seed and ten initializations. The engine computes silhouette and Davies-Bouldin indices, then creates a separate two-component PCA projection. Each cluster receives a profile of size, mean grades, mean approvals and historical outcome shares. Deterministic rules assign a descriptive name according to these summaries.

Libraries and implementation. backend/analytics/cohorts.py uses scikit-learn's scaler, KMeans, PCA and metrics, plus Pandas and NumPy. GET /api/cohorts/summary?k=4 returns the analysis; query validation allows k from two through eight. CohortAnalyticsView.tsx presents profiles and projected points.

Output and handoff. Outputs include cluster profiles, computed validation metrics and up to 400 sampled PCA points. M12 selects the largest cluster for a card. M13 compares a selected student with profile means, but does not reuse the exact fitted K-Means membership.

Educational use case and boundary. A group with low completion can motivate investigation of assessment or support needs. Cluster names are rule-based interpretations and can repeat across multiple clusters. They are not independent outcome predictions because naming uses known outcome shares. The unseeded display sample, overlapping engineered inputs and lack of cluster-stability studies limit reproducibility and substantive interpretation. No numerical clustering-quality claim is imported from the existence of a metric calculation alone.

6.11 M11 - Anomaly Analysis

Purpose and why it matters. M11 identifies combinations that are statistically unusual among the selected academic features. It complements the single-variable fences returned by M5.

Input. Nine selected numerical variables: admission grade, age, both semesters' enrolled units, approved units and grades, and signed semester grade change. The configurable contamination parameter defaults to 0.05.

Processing and method. The engine fills missing values with zero and fits an Isolation Forest with seed 42. It treats the fitted label -1 as anomalous, negates the decision score for higher-is-more-unusual record ranking, and sorts flagged records. A set of independent heuristics adds readable notes for high entry grade with low first-semester grade, high load with no approvals, or large grade shifts.

Libraries and implementation. backend/analytics/anomalies.py uses scikit-learn, Pandas and NumPy. GET /api/anomalies/summary?contamination=0.05 accepts values between 0.01 and 0.30. AnomalyAnalysisView.tsx displays the response.

Output and handoff. The response contains total records, flag count and rate, parameter value, method name, a ten-bin score distribution and a ranked list limited to 50 records. The same list is supplied through records and top_anomalies. M12 incorporates the aggregate into an anomaly card.

Educational use case and boundary. An unusual mismatch between entry score and semester performance can prompt contextual checking. A record may also be a valid exceptional achiever. The contamination setting is not an empirical failure prevalence, and heuristic notes are not model-derived attribution. Although the module comment mentions IQR and Z-score methods, the executed M11 detection path is Isolation Forest; IQR belongs to M5 and no separate Z-score detector is implemented in this function.

6.12 M12 - Educational Insights Engine

Purpose and why it matters. M12 aims to translate analytical outputs into readable findings with supporting metrics. The intended contribution is traceability from interpretation back to a source module.

Input. The shared master and selected outputs from statistical testing, progression, clustering and anomaly analysis. The function constructs six cards in a fixed order.

Processing and method. Deterministic rules combine counts and metrics with fixed text templates. The cards cover outcome composition, scholarship association, a first-semester group comparison, progression categories, the largest cluster and anomalous records. The current function does not call a generative language model.

Libraries and implementation. `backend/analytics/insights.py` orchestrates the Python engines. `GET /api/insights/summary` exposes its result to `EducationalInsightsView.tsx`. The returned engine type is Deterministic Rule.

Output and handoff. Each card includes an identifier, title, finding, category, source module, supporting metrics and interpretation. These cards are intended to guide exploration in population and individual views; the code does not automatically launch or evaluate an intervention.

Educational use case and boundary. A reliable card could report a computed progression proportion and link it to further record review. In this snapshot, the scholarship card filters for Scholarship/No Scholarship while the master contains Yes/No, resulting in `nan%` text during targeted execution. Its p-value annotation is fixed rather than formatted from the result. The first-semester t-test groups by scholarship, so it does not establish the displayed claim about future academic difficulty. Other explanations about fatigue or curriculum complexity are unsupported causal suggestions. These deficiencies are explicitly recorded; the report does not endorse all six cards as validated educational findings.

6.13 M13 - Student Digital Twin

Purpose and why it matters. M13 creates a data-driven analytical representation of an individual student record. It brings several dimensions into a common population-relative view, supporting a discussion that is more informative than a single rank or grade.

Input. A generated student identifier and the full historical master. The list endpoint returns the first requested number of records, default 100 and maximum 500, rather than providing a search over the entire dataset.

Processing and method. The engine retrieves the row, constructs an academic and demographic profile, and uses SciPy `percentileofscore` for six percentile values. Its default rank handling averages percentage rankings for tied scores. Five selected percentiles become radar axes: entry preparation, Semester 1 performance, Semester 2 performance, curricular completion and academic progression. The chart uses 50 as a conceptual percentile reference. [12]

The engine calls M10 with $k=4$, then chooses the profile minimizing the sum of absolute differences in Semester 2 grade and approved-unit count. This two-variable, unscaled nearest-profile heuristic is an approximate comparison, not the student's original eight-dimensional K-Means assignment. A fixed template describes grade change and percentile position.

Libraries and implementation. `backend/analytics/digital_twin.py` uses Pandas, SciPy and the cohort engine. Routes are `GET /api/digital-twin/students` and `GET /api/digital-twin/profile/{student_id}`. `DigitalTwinView.tsx` presents the profile and radar.

Output, use case and boundary. The result includes the profile, percentiles, radar dimensions, approximate cohort and evidence summary. A mentor can use it to compare performance with the reference population, while M14 uses it as a baseline. A percentile is a relative position, not a success probability. Invalid IDs silently return the first student's profile in the current implementation, which is a significant identity-handling defect. The twin is deterministic analytical software, not a physical simulation, live student replica or generative AI assessment.

6.14 M14 - Intervention Simulator

Purpose and why it matters. M14 supports what-if exploration: how would selected analytical indicators change if a record had different second-semester grades or approved units? It helps users understand formula sensitivity while preserving the distinction between changing inputs and causing real-world improvement.

Input. A JSON request with `student_id` and optional `new_sem2_approved`, `new_sem2_grade` and `tuition_fees_paid`. The baseline is obtained through M13.

Processing and method. The engine substitutes the requested values into a scenario. It divides proposed approved units by $\max(1, \text{enrolled units})$, clips the resulting ratio to $[0,1]$, and recalculates the weighted progression index. It compares the new values with the unchanged population distributions using percentile ranks. Entry and Semester 1 percentiles remain fixed. Before/scenario radar values and three metric deltas are assembled without altering the dataset.

Libraries and implementation. `backend/analytics/intervention_simulator.py` uses Python numerical operations, Pandas, SciPy and the digital-twin function. `POST /api/intervention-simulator/simulate` uses a Pydantic request model. `InterventionSimulatorView.tsx` displays the baseline, controls and returned comparison.

Output and handoff. The response contains baseline profile, baseline cohort name, scenario values and percentiles, radar comparison, metric deltas and a scientific disclaimer. It is intended for user interpretation and future intervention planning; it does not execute a faculty workflow.

Educational use case and boundary. A user can inspect how higher approved-unit completion changes the progression percentile while holding the reference population fixed. The current scenario does not recompute cohort membership, despite broader wording in its module comment. Tuition status is echoed in the response but does not change the financial-risk feature or the displayed academic percentiles. Domain bounds are not enforced for submitted grades or approvals, and its zero-enrollment rule differs from the master formula. This is deterministic association-based scenario analysis, not an estimated treatment effect or guarantee of graduation.

CHAPTER 7 - TECHNOLOGY STACK

7.1 Analytical and service technologies

Technology	Layer	Purpose in source	Why appropriate to this project
Python	Analytics	Implements loaders, statistical functions and module engines.	Keeps data preparation and analysis in one inspectable language.
Pandas	Data / analytics	CSV I/O, DataFrames, renaming, grouping, null checks, descriptive statistics and crosstabs.	Matches the student-level tabular representation and supports concise, traceable transformations.
NumPy	Analytics	Arrays, histogram bins, numerical operations and sample selection.	Supports efficient numerical work used alongside DataFrames and scikit-learn.
SciPy	Statistics	Pearson, Spearman, Welch, Chi-square, skewness, kurtosis and percentile calculations.	Provides established numerical implementations instead of hand-written approximations in the UI.
scikit-learn	Exploratory analytics	StandardScaler, K-Means, PCA, Isolation Forest and cluster metrics.	Supplies composable numerical transformations and exploratory algorithms with explicit parameters.
FastAPI	API	Registers analytical endpoints and serializes responses.	Exposes Python functions directly through a service boundary usable by the frontend.
Pydantic	API validation	Defines statistical and simulation request models.	Describes request fields; richer domain constraints can be added at the same boundary.
Uvicorn	Runtime	Runs the FastAPI application.	Provides the ASGI service process used by the documented Python startup path.
PyArrow / FastParquet	Data persistence	Declared Parquet engines; builder attempts Parquet export.	Support a typed columnar format for prepared analytical data. Actual writes depend on available engines.

Table 7.1. Confirmed analytical technologies and project-specific selection rationale. Python requirements are unpinned, so this table does not imply a verified installed version for every library.

The choice of Python is particularly important because the core work is educational data science. Serving calculations from Python avoids reimplementing numerical algorithms in browser components and gives the same source of formulas to multiple views. The architecture can later expose the validated feature layer to other consumers without requiring the dashboard to become the analytical engine.

7.2 Presentation and development technologies

Technology	Layer	Purpose in source	Why appropriate to this project
React	Presentation	Module views, controls and reusable components; manifest specifies ^18.3.1.	Supports an interactive interface built from independently maintained analytical views.
TypeScript	Presentation	Component and selected response types; dev dependency ^5.8.3.	Helps catch structural mistakes during development, though runtime JSON still needs validation.
Vite	Build / development	Development server, React build and /api proxy; ^5.4.19.	Connects local frontend development with the Python service and creates frontend assets.
Tailwind CSS	Styling	Utility-based layout and visual styling; ^3.4.17.	Provides consistent spacing, responsive structure and component appearance.
TanStack React Query	Server state	Queries, mutations, caching, loading and error states; ^5.83.0.	Keeps asynchronous analytical responses synchronized with view selections.
Recharts	Visualization	Bars, scatter plots, lines and radar comparisons; ^2.15.4.	Integrates charts with React and consumes the API's precomputed data structures.
Radix UI / shared UI components	Presentation	Dialogs, controls and reusable interface primitives.	Reduce repeated interface code and support consistent component behavior.
Lucide React	Presentation	Module and status icons.	Supplies a consistent icon vocabulary without changing analytical calculations.
Git / GitHub	Development	Repository metadata and repository-oriented project workflow.	Supports source history and collaboration; supplied history is not proof of deployment or CI success.
Vitest / Testing Library	Verification	Declared frontend test tools and setup.	Can verify component behavior; the supplied example test alone provides no analytical coverage.

Table 7.2. Frontend and development technologies confirmed in the archive. Version strings are manifest ranges, not a statement that these are current releases or the versions used in a successful build.

7.3 Declared and adjacent technologies

The requirements also declare Statsmodels, python-multipart and pydantic-settings. Their presence is dependency evidence, but the inspected M1-M14 functions do not justify attributing a specific additional analytical method to them. The frontend includes other utilities such as React Router, Zod, jsPDF and supporting UI packages; these are not the core statistical computation layer.

The adjacent ML Hub imports XGBoost, SHAP and joblib and has saved model artifacts. This is an extension found in the archive, not a reason to report classification accuracy as the central EduNexus result. XGBoost and SHAP are absent from the supplied requirements file despite their import through `main.py`. The report does not load the saved pickle artifacts or treat their presence as proof of a validated training experiment.

The archive also includes a Node/Express-oriented service and CIT-Sentinel AI-related code. Those resources explain the mixed historical project material; they do not establish that M12, M13 or M14 use generative AI. In the inspected UCI analytical path, insights, twin summaries and simulations are deterministic.

CHAPTER 8 - RESULTS AND DISCUSSION

8.1 Confirmed dataset and stored audit results

The following values are grounded in the supplied files. Dataset counts were independently recounted to check consistency; the cleaning values appear in the uploaded JSON audit. No new inferential result is substituted for missing experimental evidence.

Item	Confirmed value	Evidence and interpretation
Raw table	4,424 rows; 37 columns	Primary UCI CSV. The 37 includes the target.
Original explanatory attributes	36	Raw structure and official UCI metadata.
Analytical master	4,424 rows; 51 columns	Stored master CSV, consistent with ID, labels and eight features.
Graduate records	2,209	Count of the raw Target value.
Dropout records	1,421	Count of the raw Target value.
Enrolled records	794	Count of the raw Target value.
Pandas-detected null cells	0 in raw and master	Direct file inspection; does not rule out semantic missingness.
Exact raw duplicates	0	Direct check before generated identifiers.
Stored cleaning rows	4,424 before; 4,424 after	Existing cleaning audit; does not demonstrate a deletion operation.
Stored cleaning counts	0 duplicates removed; 0 missing values handled; 0 invalid records flagged	Existing audit JSON; interpreted according to the implemented checks.
Engineered features	8	Source catalog and stored columns.

Table 8.1. Verified structural and stored-audit evidence. These values establish what the supplied dataset contains, not model performance or institutional impact.

8.2 Unreported analytical results

Result category	Reporting status	Why it is not inferred
Pearson / Spearman coefficients	Not reported / requires runtime execution.	Implemented computation is not a stored result for a named variable pair.
Welch statistics, p-values and Cohen's d	Not reported / requires runtime execution.	A reproducible question, group selection and complete result record are needed.
Chi-square result and significance	Not reported / requires runtime execution.	Fixed narrative text in the insights engine is not accepted as a validated result.
Silhouette / Davies-Bouldin values	Not reported / requires runtime execution.	Values depend on the selected k, features, preprocessing and environment.
PCA explained variance	Not reported / requires runtime execution.	Two retained components do not establish a particular explained-variance percentage.
Observed anomaly count / percentage	Not reported / requires runtime execution.	Default contamination 0.05 is a configuration, not a reported observation.
Predictive accuracy, F1 or AUC	Not reported / requires runtime execution.	Supervised experiments are outside the main M1-M14 report scope and no performance claim is adopted.
Intervention benefit or dropout reduction	Not reported; requires real-world evaluation.	Recalculated indicators do not measure a causal intervention effect.

Table 8.2. Deliberately unclaimed outcomes. Targeted executions used for verification in Chapter 9 are not presented here as a new experimental results study.

8.3 What the implemented methods can reveal

Descriptive statistics can identify the center and spread of meaningful numerical variables. EDA can reveal uneven distributions or relationships worth investigating. Statistical tests can quantify compatibility with specific null hypotheses when their assumptions and variable choices are appropriate. Engineered features provide interpretable progression and completion summaries.

K-Means can expose groups with similar numerical profiles; it does not establish fixed types of students. PCA can help display high-dimensional variation but can conceal distinctions in omitted dimensions. Isolation Forest can prioritize unusual combinations for review; it does not determine which students will fail. The digital twin can show relative standing, while the simulator can explain how its formulas respond to changed inputs.

These capabilities support educational enquiry. They must not be described as demonstrated improvements in retention, a reduction in dropout, unbiased institutional decisions or real-time intervention success. Such claims require a different evaluation design and outcome evidence.

8.4 Reconciliation of documentation and code

Documentation-level idea	Source-verified behavior	Consequence for the report
All statistics are runtime evidence with no placeholders	Quality consistency and pipeline completion are fixed; some insight prose is fixed.	Qualify the claim and distinguish computed fields from configured statements.
Cleaning removes duplicates and handles missingness	M4 counts conditions and writes an audit; it does not transform the master.	Describe its current audit role and identify repair operations as future work.
Digital twin assigns a K-Means cohort	M13 selects a nearby profile using two unscaled variables.	Call it approximate profile matching.
Simulator updates cohort membership	Response retains the baseline cohort and does not return a newly assigned one.	Limit the implemented claim to indicator and percentile recalculation.
Financial what-if changes the analytical profile	Tuition flag is returned but does not change the displayed academic metrics.	Disclose the limited control effect.
Insights give fully supported explanations	Scholarship-label mismatch yields undefined percentages; some conclusions exceed the test.	Treat affected cards as defects, not valid educational findings.

Table 8.3. Material differences between intended capability and current behavior. These are opportunities for improvement rather than fabricated completed features.

8.5 Visual evidence policy

The supplied foundation PDF contains explanatory text and diagrams, while `PITCHH.pdf` is a CIT-Sentinel pitch script. No useful current UCI-dashboard screenshot or presentation slide image was identified among the supplied materials. The report therefore uses original, source-grounded architecture and workflow diagrams rather than fabricating UI screenshots or reproducing unrelated Sentinel material as EduNexus evidence.

The diagrams explain structure, not runtime success. A later institutional submission can add actual captures from a verified local run, each labeled with module, data version and parameters. Screenshots should supplement, not replace, the underlying numeric and methodological evidence.

CHAPTER 9 - TESTING AND VALIDATION

9.1 Validation approach and evidence levels

Verification for this report had three distinct levels. First, supplied files were inspected and compared: instructions, foundation PDF, raw/master CSVs, saved audit, source modules, routes and package manifests. Second, selected analytical functions were executed locally against the uploaded master to verify formulas and material limitations. Third, the report identifies API, browser and build tests that remain unexecuted. A source review or successful Python function call is not labeled as an end-to-end application test.

The extracted working copy was used for inspection. No corrective change was applied to the user's uploaded project. Saved model pickle artifacts were not loaded. The validation objective was to make the report accurate, not to silently repair and then describe a different implementation.

9.2 Executed checks

Check	Result	Scope and interpretation
Raw and master shape	Confirmed 4,424 x 37 and 4,424 x 51	Direct reads of the supplied CSV files.
Outcome counts	Confirmed 2,209 / 1,421 / 794	Counts reconcile across the raw and master representations.
Null and raw duplicate checks	Zero counted conditions	Structural snapshot check; not a full semantic audit.
Eight feature formulas	All eight matched	Recomputed from stored source fields using the implemented rounding and numerical tolerance.
Python syntax	37 backend Python files parsed	Syntax validation only; does not establish successful dependency imports.
Core analytical dependency execution	Selected twin, simulator and insight calls completed	Invoked their analytical dependencies; no HTTP or browser claim is implied.
Unchanged scenario, first record	Reported deltas were zero	One selected baseline case; does not cover all edge cases.
Invalid student identifier	Returned EDU-000001	Confirmed identity fallback defect.
Tuition-only scenario change	Academic percentiles unchanged	Confirms that this control does not affect the displayed academic profile in the selected probe.
Scholarship insight	Returned undefined percentage text	Confirmed mismatch between Yes/No source labels and the insight filters.

Table 9.1. Checks performed during preparation of this report. Functional checks expose defects as well as successful behavior; the table is not a blanket 'all tests passed' statement.

9.3 Data and analytical validation

The feature check independently rebuilt all eight expected arrays from the stored academic and financial inputs, then compared them to the corresponding master columns. This tests an important cross-file contract: the stored feature layer agrees with the documented implementation for the supplied data. It does not establish sensitivity, fairness or criterion validity of a composite indicator.

The raw duplicate check was intentionally performed before generated identifiers, resolving a weakness of the master-level audit. The class counts were verified against both representations. A deeper reproducibility study should additionally hash the raw file, version the schema, record package versions and compare a clean rebuild with the cached master.

For inferential procedures, validation must include hand-checkable or independently computed cases, the selected groups, valid sample sizes and assumption diagnostics. A test returning a number is insufficient if a categorical code was treated as a measurement or a third group was silently omitted. Cluster validation should check stability under different seeds and feature choices, while anomaly validation should examine both input quality and expert-reviewed flagged examples.

9.4 API and frontend/backend integration

The route inventory was verified statically against `main.py` prefixes and each router decorator. Frontend request strings were inspected for the corresponding analytical views. The Vite configuration confirms that `/api` requests are forwarded to port 8000 during development. These checks establish intended wiring, not successful HTTP execution.

Full API tests were not run because the local inspection environment did not contain FastAPI or HTTPX. The main application additionally imports the ML Hub, whose XGBoost and SHAP dependencies are missing from the supplied requirements. No live server response, browser interaction or production routing result is claimed. These are material validation limits, stated explicitly rather than converted into pass results.

9.5 Build verification

The project manifest defines `npm run build`, `npm run lint` and `npm run test`, and includes Vite, TypeScript and Vitest. A frontend build was not executed in this report workflow; the archive's dependency directories were excluded from the extracted inspection copy, and no installation was performed. Build outcome: **Not reported / requires runtime execution.**

The supplied `src/test/example.test.ts` checks only that true equals true. It verifies the existence of a runnable example structure, not module behavior or analytical correctness. No claim of comprehensive automated test coverage can be made from this file. Dependency pinning, clean installation and a meaningful integration test suite should precede a deployment-readiness statement.

9.6 Required edge-case and acceptance tests

Test case	Expected defensible behavior	Current evidence or required action
Empty dataset / missing required columns	Clear validation error before numerical calculations.	Add or verify explicit ingestion checks.
Duplicate original records with different generated IDs	Detect duplicates using source attributes or an approved business key.	Current master duplicate check can miss them.
Zero enrolled units	Represent undefined completion consistently.	Master and simulator use different conventions.
Unknown student ID	Return a not-found response with no substituted student.	Current fallback confirmed; correction required.
Grade outside permitted range	Reject invalid request at the API boundary.	Simulation model has optional fields without domain bounds.
Approved units above enrolled units	Reject or explicitly flag the scenario.	Ratio clipping alone can hide inconsistent inputs.
Three groups selected for Welch test	Require explicit two-group selection or another method.	Current implementation uses the first two groups.
Constant / all-missing test variable	Return a clear unavailable-result response.	Numerical and JSON handling needs testing.
Scholarship insight categories	Use actual labels and finite denominators.	Current mismatch produces undefined text.
Repeated cohort request	Stable full assignments with controlled display sampling.	K-Means seed fixed; chart sampling unseeded.
Stale processed master	Detect data/schema changes and rebuild deliberately.	No hash-based cache invalidation exists.
Production frontend request	Reach the configured backend through explicit routing.	Vite proxy only verifies development configuration.

Table 9.2. Focused acceptance tests tied to observed implementation risks. These are proposed gates, not tests reported as executed.

9.7 Reproducibility procedure

A reproducible local evaluation should first record the project snapshot and raw-file checksum. Next, create an isolated Python environment, reconcile the declared dependencies with all imported modules, and preserve package versions. Rebuild the analytical master from the known raw input, retaining an original copy for comparison. Record the schema, row counts and engineered-feature formula checks.

Start the FastAPI service from the backend directory using the documented Uvicorn entry point, then verify `/health`, a representative dataset route, a statistical request and a scenario request. Start the frontend with the project's development script and inspect the corresponding UI responses through the configured proxy. After these checks, run the frontend build and meaningful tests. Preserve the selected columns, k , contamination, random seeds and returned metrics for each analytical result.

This procedure turns a future results chapter into a reproducible experiment log. It also separates data-version changes from algorithm changes and frontend presentation changes. None of these future execution steps is represented as already completed in this report.

CHAPTER 10 - LIMITATIONS AND FUTURE SCOPE

10.1 Dataset and generalization limitations

EduNexus currently operates on a historical research dataset. The attributes, course context, admission routes and outcome recording practices may differ from those of another institution. A result observed here cannot automatically be generalized to students at Chennai Institute of Technology, all engineering colleges or the wider student population.

The dataset contains observed outcomes and two semester summaries. It does not supply a live institutional process, current attendance stream or continuous learner activity history in the primary analytical workflow. Limited temporal resolution restricts questions about the timing of decline. Potential institutional selection effects and unmeasured circumstances also affect interpretation.

The three outcome groups should be preserved carefully. Enrolled students are not necessarily eventual graduates or dropouts, and their observed status should not be converted into an assumed final outcome. Further institutional validation needs definitions of observation time, completion horizon and the available information at each decision point.

10.2 Statistical and methodological limitations

Correlation and group association do not establish causality. Multiple exploratory tests can increase false-positive findings without correction. Current hypothesis-test routes do not provide complete assumption checks, confidence intervals or multiple-testing adjustment. Small or degenerate groups need explicit handling, and the independent-samples t-test is unsuitable for repeated measurements from the same students.

K-Means depends on feature selection, scaling and k. Its approximately compact distance-based clusters may not capture every meaningful educational grouping. Correlated raw and derived fields can repeatedly emphasize the same academic signal. PCA is a projection rather than an explanation of individual outcomes. Isolation Forest scores reflect unusualness in selected dimensions and depend on a user-controlled threshold.

The progression index, six-unit load reference and half-point trend boundary are transparent heuristics. Their educational interpretation requires empirical and domain validation. An analytical digital twin is a summary of available records, and a scenario recalculation is not a causal intervention model. None of these outputs justifies an automatic high-stakes student decision.

10.3 Software and evidence limitations

The current cleaner audits rather than repairs the master. Quality consistency and module-ready statuses are fixed values. Lineage records lack execution-aware provenance. The cache can reuse outdated data. Student identifiers depend on row order, and the twin substitutes the first record when an identifier is unknown. Insight templates include a label mismatch and unsupported interpretations. Scenario inputs lack sufficient domain bounds, and financial changes do not alter the returned academic indicators.

Full API, browser and build verification was not completed in the report environment. The current backend dependency declaration also omits imports needed by its eagerly loaded ML extension. These implementation and validation gaps should be addressed before any claim of production readiness. The presence of old project assets and an adjacent ML Hub further emphasizes the need to define a clear supported application entry point.

10.4 Privacy and institutional use

Generated identifiers reduce direct name exposure, but demographic and academic combinations may still identify people in a real institutional setting. A pseudonymous ID alone is not an anonymization guarantee. Institutional integration should apply purpose limitation, minimum necessary data, access control, retention rules and auditable access.

No authentication or authorization dependencies were identified on the inspected core analytics routers. The existence of separate login code elsewhere in the archive does not prove that these routes are protected. The permissive development CORS configuration also needs review for deployment. Privacy and fairness should be evaluated in the actual usage context, including whether sensitive attributes are necessary for a particular analysis and how staff communicate findings to students.

10.5 Prioritized future extensions

Extension	Proposed work	Evidence required before claiming completion
1. Real institutional data integration	Map authorized institutional sources into the canonical schema with persistent entity keys and validated semantics.	Documented mappings, representative field checks and consent/access arrangements.
2. Automated ETL pipelines	Schedule ingestion, validate schemas, log applied cleaning actions and version processed outputs.	Repeatable runs with row-count reconciliation, hashes and failure recovery.
3. Real-time analytics	Add timestamped event sources and incremental processing where a defined decision requires it.	Measured freshness, event-time semantics and tested update behavior.
4. Predictive ML layer	Let a separate SENTINAL D / EduPredict AI system consume validated features available at the intended prediction time.	Leakage-safe evaluation, calibration, suitable class metrics and external validation.
5. MLOps	Track experiments, training data versions, feature versions and model artifacts for the optional prediction layer.	Reproducible experiment records and a controlled release process.
6. Model monitoring	Monitor drift, calibration, subgroup behavior and operational failures after a model is deployed.	Defined thresholds, alerts and a documented response workflow.
7. Automated intervention workflows	Connect reviewed findings to authorized staff tasks and record intervention delivery.	Human oversight, event logs and evaluation of actual outcomes.
8. Institutional deployment	Package services, configure routing, isolate extensions and establish recovery and observability.	Successful builds, API/browser tests and representative load checks.
9. Privacy and security improvements	Add route authorization, minimize data exposure and manage secrets and retention.	Access-control tests and institutional review appropriate to the data use.
10. Academic information-system integration	Build stable adapters to supported student information and learning systems.	Contract tests, ownership rules and reconciliation of duplicated or late records.

Table 10.1. Future scope with concrete implementation and validation expectations. These are proposed extensions, not features claimed as deployed.

10.6 Recommended implementation order

The first priority is correctness of the existing analytical contract: fix scholarship labels and unsupported insight language, reject unknown student IDs, enforce scenario bounds, remove fixed quality claims, and implement or accurately rename cleaning behavior. The next priority is reproducibility: define dataset versions, pin dependencies, invalidate stale caches, seed visualization sampling and preserve parameterized result records.

Only after these foundations are reliable should the project expand toward live data or supervised prediction. A separate predictive layer should use only features available at its prediction time, fit preprocessing inside training folds, handle class imbalance appropriately and assess calibration and subgroup performance. Existing ML Hub code is acknowledged as an extension in the archive; it is not merged with the main educational data science contribution or treated as validated SENTINAL D performance.

Intervention effectiveness ultimately requires an evaluation of actual support actions and outcomes, with a defensible comparison design. A higher simulated percentile cannot substitute for that evidence. This staged progression preserves EduNexus's role as the explainable data foundation for later systems.

CHAPTER 11 - CONCLUSION

EduNexus implements a multidimensional educational data science framework around the UCI Predict Students' Dropout and Academic Success dataset. Its central contribution is the connection of data understanding, quality assessment, descriptive and inferential analysis, feature engineering, progression comparison, cohort discovery and anomaly inspection within a shared analytical architecture. The frontend presents these computations; it does not define the core statistical methods.

The supplied primary dataset contains 4,424 records and 37 original columns, including the target. The prepared master contains 51 columns after adding a generated identifier, readable labels and eight engineered features. The report verified this structure, reconciled the outcome counts and confirmed that all eight stored features agree with the implemented formulas. The methods and API inventory are grounded in the supplied source rather than inferred from module names or promotional descriptions.

The fourteen modules support complementary views of educational information. Descriptive statistics explain distributions; EDA exposes candidate patterns; formal tests address specific associations; progression measures compare two observed semesters; K-Means and Isolation Forest support exploratory grouping and review. The Student Digital Twin and Intervention Simulator add population-relative individual inspection and deterministic scenario recalculation without claiming causal intervention effects or generative intelligence.

The framework also has clear boundaries. The cleaning interface is predominantly an audit, some quality and status values are configured constants, several insight statements require correction, and the twin and simulator have identity and validation limitations. Full service, browser and build verification remains necessary. These findings do not justify invented accuracy, significance or retention improvements; they identify what must be improved before institutional use.

EduNexus is most defensible as an explainable educational analytics foundation whose calculations can be traced and questioned. With stronger data contracts, reproducible validation, corrected interpretations and appropriate institutional governance, it can support future integration and a separately evaluated predictive layer. Its academic value lies in demonstrating how educational evidence is constructed from data, while stating honestly what that evidence can and cannot establish.

REFERENCES

- [1] Project requirements, **Pasted text(20260922-150328).txt**, user-supplied instructions, received 22 September 2026. Defines report scope, required chapters, eight features and the UCI source-of-truth rule.
- [2] **EDUNEXUS - NEW DOC.pdf**, "EduNexus - Master Documentation Foundation," 21 pages, user-supplied project document. Used for project identity and intended analytical narrative; reconciled against current code where claims differ.
- [3] **EDUNEXUS.zip**, user-supplied source snapshot, received 22 September 2026. Primary implementation evidence: outer project backend/data/, backend/analytics/, backend/api/, backend/main.py, src/components/views/, package.json, vite.config.ts, Python requirements, primary UCI CSV, analytical master CSV and cleaning audit JSON. The embedded project copy and CIT-Sentinel/OULAD materials are treated as historical or adjacent evidence, not the current primary data path.
- [4] UCI Machine Learning Repository, "Predict Students' Dropout and Academic Success," Dataset 697, donated 2021. Dataset record and variable metadata. Accessed 22 September 2026. <https://archive.ics.uci.edu/dataset/697/predict+students+dropout+and+academic+success>
- [5] SciPy Developers, "scipy.stats.pearsonr," SciPy API Reference. Accessed 22 September 2026. <https://docs.scipy.org/doc/scipy/reference/generated/scipy.stats.pearsonr.html>
- [6] SciPy Developers, "scipy.stats.spearmanr," SciPy API Reference. Accessed 22 September 2026. <https://docs.scipy.org/doc/scipy/reference/generated/scipy.stats.spearmanr.html>
- [7] SciPy Developers, "scipy.stats.ttest_ind," SciPy API Reference. Used to verify the Welch option and independent-sample interpretation. Accessed 22 September 2026. https://docs.scipy.org/doc/scipy/reference/generated/scipy.stats.ttest_ind.html
- [8] SciPy Developers, "scipy.stats.chi2_contingency," SciPy API Reference. Accessed 22 September 2026. https://docs.scipy.org/doc/scipy/reference/generated/scipy.stats.chi2_contingency.html
- [9] scikit-learn Developers, "KMeans," scikit-learn API Reference. Accessed 22 September 2026. <https://scikit-learn.org/stable/modules/generated/sklearn.cluster.KMeans.html>
- [10] scikit-learn Developers, "PCA," scikit-learn API Reference. Accessed 22 September 2026. <https://scikit-learn.org/stable/modules/generated/sklearn.decomposition.PCA.html>
- [11] scikit-learn Developers, "IsolationForest," scikit-learn API Reference. Accessed 22 September 2026. <https://scikit-learn.org/stable/modules/generated/sklearn.ensemble.IsolationForest.html>
- [12] SciPy Developers, "scipy.stats.percentileofscore," SciPy API Reference. Accessed 22 September 2026. <https://docs.scipy.org/doc/scipy/reference/generated/scipy.stats.percentileofscore.html>

Source interpretation note

External references establish dataset identity and method semantics; they are not evidence that a particular EduNexus result was achieved. Software documentation can change, so the reported project parameters come from the uploaded code. The numerical examples in the foundation document and the projected outcomes in the separate CIT-Sentinel pitch are not adopted as experimental EduNexus results.

APPENDIX A - VERIFIED API INVENTORY

The paths below combine the prefixes in `backend/main.py` with the decorators in each corresponding router. They are source-verified routes; runtime availability has not been asserted. Unless specified as a POST request, these routes retrieve or calculate a JSON response. [3]

Module	Method and actual endpoint	Contract
Service health	GET /health	Fixed service metadata, version and dataset name.
Pipeline metadata	GET /api/pipeline/status; GET /pipeline/status	Fixed ready/complete entries for M1-M14; not an executed readiness probe.
M1	GET /api/datasets/status	Dataset availability and extended-source metadata.
M1	GET /api/datasets/summary	Master profiling and column descriptions.
M2	GET /api/integration/summary	Current master counts and compatibility status.
M2	POST /api/integration/run	Semantic preparation summary and six descriptive lineage steps.
M3	GET /api/quality/summary	Completeness, uniqueness, domain issues and configured composite.
M4	GET /api/cleaning/summary	Read or generate cleaning audit JSON.
M4	POST /api/cleaning/run	Execute audit calculations and attempt audit-file write.
M5	GET /api/statistics/columns	Numeric and categorical field lists.
M5	GET /api/statistics/variable/{col_name}	Selected-variable statistics and distributions.
M5	GET /api/statistics/summary	Overall outcome and selected academic averages.
M6	GET /api/eda/summary	Correlation matrix, grouped summaries and samples.
M6	GET /api/eda/bivariate	Required query x and y; group defaults to target.
M7	GET /api/statistical_analysis/columns	Available fields for analysis.
M7	POST /api/statistical_analysis/run	JSON body: test_type, var1, var2.
M8	GET /api/features/summary	Eight-feature catalog and descriptive distributions.
M9	GET /api/temporal/summary	Two-semester progression aggregates.
M10	GET /api/cohorts/summary	Query k: default 4, allowed 2-8.
M11	GET /api/anomalies/summary	Query contamination: default 0.05, allowed 0.01-0.30.

Module	Method and actual endpoint	Contract
M12	GET /api/insights/summary	Six deterministic cards; known interpretation defects apply.
M13	GET /api/digital-twin/students	Query limit: default 100, allowed 1-500.
M13	GET /api/digital-twin/profile/{student_id}	Profile, percentiles and approximate cohort comparison.
M14	POST /api/intervention-simulator/simulate	JSON student_id plus optional new_sem2_approved, new_sem2_grade and tuition_fees_paid.

Table A.1. Complete source-verified M1-M14 route inventory plus service metadata endpoints. Adjacent ML Hub routes are intentionally excluded from the core inventory.

Representative request contracts

A statistical request can specify `test_type = chi-square`, `var1 = scholarship_label` and `var2 = target`. This selects a categorical association question. The exact computed statistic and p-value must be recorded from a validated run; neither is supplied as an assumed result here.

A simulation request must identify a known student and can provide a proposed second-semester grade or approved-unit count. Omitted fields use the baseline. The response contains the baseline, scenario, radar comparison, metric deltas and disclaimer. The current API accepts typed values but does not enforce all educational bounds, so a typed request is not necessarily a valid scenario.

APPENDIX B - SOURCE TRACEABILITY

Report claim or module	Primary source location	Evidence role
Dataset identity and source fields	backend/data/uci_loader.py; backend/data/schema.py	Defines active CSV path, raw mapping and variable semantics.
Raw counts and target distribution	data/raw/uci_predict_students_dropout_and_academic_success.csv	Direct observation-level evidence.
Enriched structure and stored features	data/processed/master_analytical_dataset.csv	Confirms 51 columns and eight stored transformations.
M1 / M2	backend/data/loaders.py; semantic_mapper.py; integrator.py	Profiling, semantic preparation and descriptive lineage.
M3 / M4	backend/analytics/quality.py; cleaning.py	Actual checks, configured quality component and audit behavior.
Saved cleaning values	data/processed/cleaning_audit_log.json	Existing project audit output.
M5 / M6 / M7	backend/analytics/descriptive.py; eda.py; statistical_tests.py	Statistical formulas, sampling and hypothesis-test behavior.
M8 / M9	backend/analytics/feature_engineering.py; progression.py; temporal.py	Feature catalog and progression thresholds.
M10 / M11	backend/analytics/cohorts.py; anomalies.py	Feature lists, seeds, clustering, PCA and detection parameters.
M12 / M13 / M14	backend/analytics/insights.py; digital_twin.py; intervention_simulator.py	Deterministic narratives, percentile profiles and scenario behavior.
Endpoint inventory	backend/main.py and backend/api/*.py	Prefixes, HTTP methods, request models and query constraints.
Frontend module integration	src/components/views/*View.tsx	Requests, query state, charts and presentation responsibilities.
Dependencies / build / proxy	package.json; backend/requirements.txt; vite.config.ts	Declared libraries, scripts and local API routing.
Limited supplied test evidence	src/test/example.test.ts	Demonstrates that the example test is not functional coverage.

Table B.1. Audit trail from report topics to uploaded source files. Paths refer to the outer EDUNEXUS directory. Similar filenames in the embedded historical copy are not interchangeable evidence.

Final verification checklist

Dataset identity, raw count, original-column interpretation, target counts and enriched shape have been reconciled. All fourteen requested modules, eight implemented feature formulas, principal statistical methods, current API endpoints and confirmed technologies are documented. Current behavior is separated from proposed scope, fixed values are not presented as measured evidence, and no classifier accuracy, statistical significance or intervention benefit has been invented.